

anoX Messenger V1 — Master Development Archive

Independent architecture, security, workflow, prompt and project-state archive

anoX Messenger V1 — MASTER CONTEXT

Purpose

This is the independent continuation source for anoX Messenger V1.

Roles

ChatGPT: architecture owner/coordinator/reviewer and prompt author.

Devin: coding/testing implementation agent.

Product

Privacy/security-focused Android E2EE messenger, with GrapheneOS as an important target platform.

V1 hard decisions

- No account recovery.
- Single device per account.
- Server never receives message plaintext.
- E2EE private keys remain on device.
- Key verification required.
- Push contains no message plaintext.
- Attachments are separately encrypted.
- No custom cryptography.
- V1 does not promise complete metadata anonymity.
- Do not claim “unhackable”.

Crypto direction

Rust + vodozemac/Olm-compatible cryptography behind a narrow Android native boundary.

Current implementation

Initial crypto foundation was reported as implemented by Devin.

Critical blocker

Session decryption / OlmMessage handling was reported as non-functional.

Current next task

Fix and verify the crypto foundation before proceeding to backend/messaging.

Architecture principle

Accepted architecture

↓

Devin prompt

↓

Implementation

↓

Tests

↓

Devin report

↓



Decision



Archive update

Evidence policy

This archive distinguishes established facts from planned work and missing history. Never convert an inference into a historical fact.

Documents

See the numbered directories for requirements, architecture, security, crypto, protocols, backend/database, development, ADRs, prompts, state and historical recoverability.

Product Requirements — anoX Messenger V1

ESTABLISHED

- Android messenger.
- Security/privacy focused.
- End-to-end encrypted messaging is the central security property.
- GrapheneOS compatibility is an important target.
- V1 is single-device per account.
- V1 has no recovery mechanism.
- Server must not receive message plaintext.
- E2EE private keys must remain on-device.
- Key verification is required.
- Push payloads must not contain message plaintext.
- Attachments are treated separately from message encryption.
- V1 does not claim complete metadata anonymity.
- "Unhackable" is not a technically supportable product guarantee.

V1 OUT OF SCOPE

- Account recovery.
- Multi-device account operation.
- Community/forum features.
- Any server-side plaintext message processing.

PRODUCT SECURITY CLAIM BOUNDARY

The defensible V1 security claim is that compromise of backend/database/transport alone must not be sufficient to decrypt E2EE message content, assuming the endpoint and implementation are not compromised.

Endpoint compromise remains outside what E2EE alone can prevent.

System Architecture

Target structure

Android Client

■■■■ UI / Compose

■■■■ Domain / application logic

■■■■ Data / networking

■■■■ Secure local storage

■■■■ Narrow Crypto Interface

↓

Rust

↓

vodozemac / Olm

Android Client

↓ HTTPS + device authentication

Backend Service

↓

Supabase / PostgreSQL

↓

Encrypted message / attachment storage

Trust boundaries

Trusted endpoint components

- Application code required to handle plaintext.
- Crypto implementation.
- Local protected key material.

Untrusted infrastructure

- Network transport (beyond authenticated cryptographic protection).
- Backend with respect to message plaintext.
- Database/storage with respect to E2EE secrets.

Architectural principle

The backend is a routing/storage/authentication infrastructure component, not the holder of message plaintext.

Existing baseline

The original app was a minimal Android Compose application. The project is to be continued, not replaced with an unrelated new app.

Reported baseline:

- Android module.
- Kotlin 1.9.20.
- AGP 8.13.2.
- Gradle 9.3.1.
- compileSdk 34.
- targetSdk 34.
- minSdk 26.
- Compose BOM 2023.10.01.
- no existing crypto/backend/database/messaging implementation at audit time.

These tool versions are historical snapshots and must be revalidated before dependency-sensitive work.

Security Invariants — V1

These are non-negotiable unless explicitly changed through an approved ADR.

1. E2EE private keys never leave the device.
2. The server never receives message plaintext.
3. The server never receives E2EE private keys.
4. Push payloads contain no message plaintext.
5. V1 contains no account recovery secret or recovery device.

6. Loss of the only device/local crypto state means loss of the account cryptographic identity.
7. V1 is single-device.
8. Identity-key changes must be detectable.
9. Revoked devices must not authenticate.
10. No custom cryptographic primitives.
11. No invented cryptographic APIs.
12. Crypto failures must fail safely.
13. Secrets/private keys/plaintext must not be written to logs or crash reports.
14. Protected local crypto state needs confidentiality AND integrity/authenticity.
15. Duplicate/replayed requests/messages must be handled safely.
16. Security-sensitive dependencies and APIs must be verified against current authoritative documentation.
17. Passing application tests is not proof of cryptographic security.
18. GrapheneOS compatibility must not be confused with a guarantee against endpoint compromise.

Threat Model — V1

Relevant threats

- Database dump.
- Backend compromise.
- Network interception/MITM.
- Replay.
- Identity-key replacement.
- Stolen device.
- Local storage disclosure.
- Malicious or manipulated client requests.
- Metadata analysis.
- Supply-chain/dependency compromise.
- Corrupted/tampered ciphertext.
- Device/account revocation abuse.

Fundamental limitation

If an endpoint is compromised, an attacker can potentially observe plaintext before encryption or after decryption. E2EE does not solve endpoint compromise.

V1 metadata boundary

E2EE protects content, not all metadata. Operational metadata such as timing, routing and identifiers may remain observable.

Cryptographic Architecture

Direction

ESTABLISHED DIRECTION: Rust crypto layer using vodozamac/Olm-compatible primitives.

Devin reported selecting vodozamac 0.10.0. This version must be revalidated against current authoritative documentation before being treated as a current dependency decision.

Layering

Kotlin / Android

↓

Narrow JNI/native boundary

↓

Rust crypto layer

↓

vodozemac

Intended separation

The application should not manipulate internal vodozemac structures directly.

The crypto boundary should expose only the operations actually required by the application, such as:

- identity creation;
- public identity export;
- session establishment;
- encryption;
- decryption;
- protected state serialization/deserialization;
- secure lifecycle/destruction.

The exact function signatures must come from the verified library API. They must not be invented.

Reported first implementation

Devin reported:

- crypto/rust/Cargo.toml
- crypto/rust/src/lib.rs
- error.rs
- identity.rs
- session.rs
- serialization.rs
- Android CryptoNative.kt
- CryptoError.kt
- CryptoResult.kt
- CryptoBridge.kt

It also reported 16 JNI functions, pointer-based handles and buffer-based I/O.

CRITICAL STATUS

Session decryption was reported as non-functional because the implementation could not correctly handle/parse the vodozemac OlmMessage/session flow.

Therefore this crypto foundation is not accepted as production-complete.

Required next crypto acceptance test

A real sender/recipient round trip must succeed:

Alice identity/session

↓

encrypt plaintext

↓

ciphertext

↓

Bob session

↓

decrypt

↓

original plaintext

Negative tests must also prove that manipulated/wrong ciphertext/session material fails safely.

Key Lifecycle

ESTABLISHED DESIGN PRINCIPLES

Key material is separated by purpose.

At minimum distinguish:

1. E2EE identity key material.
2. E2EE session/ratchet state.
3. Local state-protection key.
4. Device-authentication key.
5. Authentication/session tokens.

Do not collapse unrelated secrets into one generic master key.

Android Keystore

Android Keystore is intended to protect local key material used to protect application state.

Important: Keystore does not mean arbitrary vodozamac objects are themselves stored directly in Keystore.

The implementation must document:

- what is in Keystore;
- what encrypted state is in app storage;
- integrity protection;
- corruption handling;
- deletion behavior;
- wipe/uninstall behavior.

OPEN / TO VERIFY

The exact lifecycle for every Olm/vodozamac key type, pre-key/one-time-key handling, ratchet state, rotation and expiry was not fully recoverable from the current historical context. Do not invent those details. Resolve them against the actual vodozamac API and accepted protocol design before implementation.

Local Crypto State Protection

Devin's initial implementation reportedly used AES-256-CBC for serialized crypto state with an Android Keystore master key.

Security review requirement

CBC by itself does not provide authentication/integrity.

The final construction must provide:

- confidentiality;
- integrity/authenticity;
- corruption/tamper detection.

Do not create a custom encryption construction. Prefer an established authenticated-encryption construction supported by an appropriate maintained library/API.

The exact replacement is OPEN until verified against the actual implementation/dependencies.

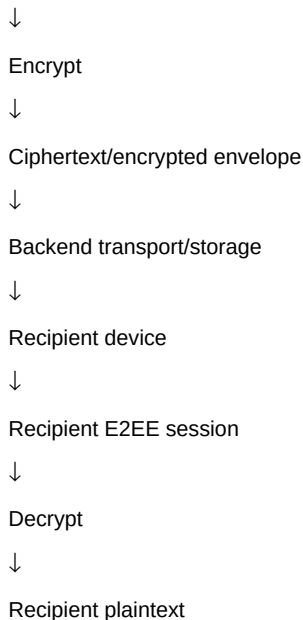
Message Flow

Core flow

Sender plaintext

↓

Sender E2EE session



Trust boundary

The crypto library runs on the endpoint and therefore necessarily processes plaintext at the moment of encryption/decryption. This does NOT mean the backend gets plaintext.

The distinction is:

Plaintext → local crypto library → ciphertext → network/server

network/server → ciphertext → local crypto library → plaintext

Requirements

- Server stores/routes ciphertext, not message plaintext.
- Tampered ciphertext must fail.
- Wrong session/key must fail.
- Duplicate/replayed operations must be safely handled.
- Message identifiers and delivery state must be designed without exposing unnecessary data.

OPEN

The exact wire envelope, message schema, ratchet/key update details and replay-window rules were not fully recoverable from current historical context and must be specified before production messaging implementation.

Key Verification

ESTABLISHED

Key verification is required in V1.

Goals

- Establish trust in the intended contact's cryptographic identity.
- Detect unexpected identity-key changes.
- Make trust changes visible to users.

Important boundary

Verification does not prove that a device is malware-free. It establishes a cryptographic identity relationship under the selected protocol.

OPEN

The exact user interaction, SAS/QR representation and vodozamac API mapping were not fully preserved in the currently recoverable historical context. They must be specified and tested before implementation.

Backend Architecture

Target

Android

↓

Authenticated HTTPS API

↓

Backend service

↓

Supabase / PostgreSQL

Responsibilities

Backend may handle:

- authentication/device authorization;
- routing;
- encrypted message storage;
- delivery state;
- rate limiting/abuse controls;
- synchronization;
- metadata required for operation.

Backend must not handle:

- message plaintext;
- E2EE private keys.

Security

Application authorization and PostgreSQL RLS are complementary.

Do not rely on RLS alone as the complete security model.

Database Architecture

ESTABLISHED DIRECTION

Supabase/PostgreSQL is the planned persistence infrastructure.

Reproducibility rule

Database structure should be created from version-controlled migrations/schema code, not by manually constructing production tables one column at a time.

That includes:

- tables;
- columns;
- types;
- constraints;
- indexes;
- relationships;
- RLS policies;
- functions/triggers where required.

CURRENT STATUS

No database schema had been implemented at the time of the codebase audit.

OPEN

Exact final tables/columns/indexes/RLS rules were not fully recoverable from the current historical context. Do not fabricate a final schema.

Devin Workflow

Roles

ChatGPT = architecture/coordinator/reviewer.

Devin = implementation/test/reporting agent.

Required cycle

1. Read repository.
2. Read relevant archive/security invariants.
3. Compare current implementation with target.
4. Implement only approved scope.
5. Run real tests.
6. Report exact files, APIs, dependencies and test results.
7. Report blockers.
8. Stop on unresolved security-critical ambiguity.
9. ChatGPT reviews.
10. Accept/reject/change.
11. Update archive and project state.
12. Generate next prompt.

Devin must not

- invent crypto APIs;
- silently change architecture;
- add recovery;
- send plaintext to server;
- log secrets/plaintext;
- assume old dependency versions are current;
- call a crypto layer secure just because compilation/tests pass.

Prompt persistence

Every substantive Devin prompt should be saved under 09_prompts/ and linked to:

- resulting implementation/report;
- relevant ADRs;
- Git commit(s), when available;
- project state.

Testing Strategy

Required categories

- Unit tests.
- Crypto round-trip tests.
- Negative crypto tests.
- Integration tests.
- Backend/API tests.
- Database/RLS tests.
- Android instrumentation tests.
- GrapheneOS device tests.
- Security regression tests.

- Dependency/security review.

Critical crypto acceptance

A successful build is insufficient.

At minimum:

- encrypt/decrypt round trip;
- wrong session/key failure;
- modified ciphertext failure;
- serialization/deserialization;
- state corruption detection;
- identity-key change detection;
- device authentication/revocation tests once implemented.

ADR-001 — No Account Recovery in V1

Status: ACCEPTED

Decision

V1 has no account recovery.

Consequence

If the only device and local cryptographic state are lost, the cryptographic account identity cannot be restored.

Explicitly excluded

- recovery seed;
- recovery phrase;
- recovery device;
- server-side private-key backup;
- hidden recovery master key.

Future

Recovery can be considered as a separate future architecture, but must not appear implicitly in V1.

ADR-002 — Rust + vodozemac Crypto Direction

Status: ACCEPTED DIRECTION

Decision

Use a Rust crypto layer with vodozemac/Olm-compatible cryptographic functionality as the E2EE foundation.

Reason

Use an established implementation rather than implementing a custom cryptographic protocol/primitives.

Consequence

Android communicates through a narrow native boundary.

Caveat

The exact dependency version and APIs must be revalidated before production use.

ADR-003 — Server Never Receives Message Plaintext

Status: ACCEPTED

Decision

Message plaintext is processed on endpoints. The backend handles ciphertext/encrypted envelopes.

Consequence

A backend/database compromise alone should not reveal message contents.

Limitation

This does not protect plaintext on a compromised endpoint.

PROMPT-002 — Crypto Foundation

Status: HISTORICAL RESULT RECOVERED; EXACT ORIGINAL PROMPT NOT FULLY RECOVERABLE

Known task intent:

- research current vodozemacs;
- establish Rust crypto layer;
- expose narrow Android/JNI boundary;
- integrate Android Keystore for protected local state;
- create identity/session abstractions;
- implement safe error mapping;
- document security review;
- do not implement backend/messaging yet.

Reported Devin result:

- vodozemacs 0.10.0 selected;
- Rust directory and modules created;
- JNI interface with 16 crypto functions;
- identity/session wrappers;
- encrypted state serialization;
- Android Keystore master key;
- Kotlin bridge/error/result classes;
- security review and completion report.

Critical blocker:

Session decryption/OlmMessage handling was reported non-functional.

NEXT PROMPT — Resolve Crypto Foundation Blocker

Status: APPROVED NEXT TASK

Scope

1. Inspect the actual current crypto implementation.
2. Verify the exact installed vodozemacs API/version.
3. Correct OlmMessage/session handling.
4. Implement a real sender/recipient encrypt/decrypt round trip.
5. Add negative tests for modified ciphertext and wrong session/key.
6. Review serialized-state protection for integrity/authenticity.
7. Harden the JNI boundary.
8. Verify no recovery mechanism exists.
9. Update crypto security documentation.

Explicitly DO NOT implement yet

- backend;
- database;

- messaging UI;
- push;
- attachments;
- account recovery.

Stop condition

If the library/API cannot support the intended operation as currently designed, stop and report the exact incompatibility instead of inventing a workaround.

anoX Messenger V1 — Current Project State

Date: August 2026

Overall

Architecture: ESTABLISHED DIRECTION

Implementation: EARLY FOUNDATION

Completed / reported

- Initial Android application exists and runs on GrapheneOS.
- Codebase audit completed.
- V1 architecture established.
- No-recovery requirement established.
- Crypto foundation implementation reported by Devin.
- Rust/vodozamac direction established.
- Android Keystore integration reported.
- Security review documentation reported.

Current blocker

CRITICAL: session decryption / vodozamac OlmMessage handling is not functional according to Devin's latest report.

Not yet implemented

- production authentication;
- device registration;
- backend API;
- database schema;
- RLS;
- messaging flow;
- key verification UI/protocol implementation;
- push;
- attachments;
- comprehensive security tests.

Next task

Resolve and validate the crypto foundation before building messaging/backend assumptions on top of it.

Recovery

Explicitly excluded from V1.

Important

This state reflects the recoverable project context, not an independent verification of the current repository. The repository must be checked before each implementation step.

Historical Recoverability Register

This file exists to prevent false historical claims.

Recoverable from current context

- anoX Messenger V1 project direction.
- Existing minimal Android app baseline.
- V1 no-recovery rule.
- Single-device rule.
- Server-no-plaintext rule.
- On-device E2EE private keys.
- Rust + vodozemacs direction.
- Android Keystore role.
- Key verification requirement.
- Device authentication separation.
- Supabase/PostgreSQL backend direction.
- Push plaintext prohibition.
- Separate attachment encryption direction.
- Security invariants listed in this archive.
- Devin implementation workflow.
- Initial crypto foundation reported result.
- Current crypto decryption blocker.

Not fully recoverable

- Exact verbatim text of every historical Devin prompt.
- Full content of the referenced "Raw1.0" architecture.
- Exact complete historical database schema, because it was not implemented and the full historical specification is not available in current recoverable context.
- Exact final key lifecycle details.
- Exact final message wire format.
- Exact final verification UX/protocol details.
- Any decision that exists only in a missing/omitted historical message.

Rule

Missing historical content must be recovered from the original chat/repository if available. It must not be invented from assumptions.

Raw1.0 — Recovery Record

Status: HISTORICAL-MISSING

The project history references a prior architecture/version called "Raw1.0".

The current recoverable context does not contain the full Raw1.0 document or complete verbatim content.

Therefore:

- Raw1.0 is acknowledged as a potentially authoritative historical source.
- No missing Raw1.0 details are fabricated here.
- If the original Raw1.0 export/file/chat content becomes available, it must be compared against this archive.
- Conflicts must be resolved explicitly through ADRs.